



endingo

정훈이의 공부일지

CATEGORY



👋 안녕하세요, AI 개발자 **이정훈**입니다

AI · 백엔드 · 데이터 분석 중심의 프로젝트와 경험을 공유합니다.

Contact Me

Data science/이미지처리

[이미지처리] 3. CIFAR-10 분류 모델링

endingo 2025. 4. 9. 12:30 ⋮

1. 데이터 다운로드

```
train_dataset = datasets.CIFAR10(root=".", train=True, download=True, transform=ToTensor())
test_dataset = datasets.CIFAR10(root=".", train=False, download=True, transform=ToTensor())
```

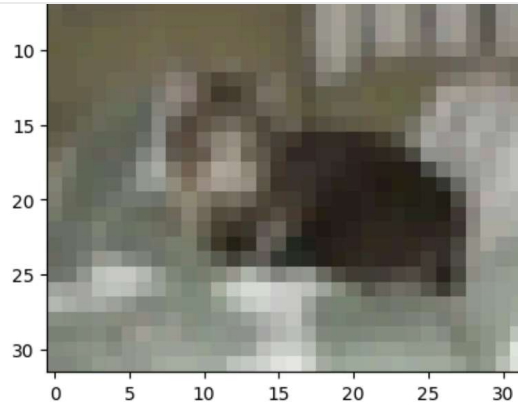
2. 데이터 이미지 확인

```
n = 150
image, label = train_dataset.data[n], train_dataset.targets[n]
```

```
# 이미지 시각화
plt.imshow(image)
plt.title(f"Label: {train_dataset.classes[label]}")
plt.show()
```



정훈이의 공부일지



3. 모델링

1) 모델선언

```
nn.Conv2d(3, 32, kernel_size=3, stride=1, padding=1)
```

: 사진이 컬러임으로 채널을 1->3으로 바꾼다.

★ 딥러닝 구조: $3 \times 32 \times 32 \Rightarrow 32 \times 32 \times 32 \Rightarrow 32 \times 16 \times 16 \Rightarrow 64 \times 16 \times 16 \Rightarrow 64 \times 8 \times 8 \Rightarrow 128 \Rightarrow 10$

```
n_class=10
```

```
dropout_rate = 0.5
```

```
model = nn.Sequential(
    nn.Conv2d(3, 32, kernel_size=3, stride=1, padding=1), nn.ReLU(), nn.Dropout2d(p=dropout_rate), # 첫 번째
    드롭아웃
    nn.MaxPool2d(kernel_size=2),
    nn.Conv2d(32, 64, kernel_size=3, stride=1, padding=1), nn.ReLU(), nn.Dropout2d(p=dropout_rate), # 첫 번째
    드롭아웃
    nn.MaxPool2d(kernel_size=2),
    nn.Flatten(),
    nn.Linear(64*8*8, 128), nn.ReLU(), nn.Dropout(p=dropout_rate), # 첫 번째 드롭아웃
    nn.Linear(128, n_class)
).to(device)
loss_fn = nn.CrossEntropyLoss()
optimizer = Adam(model.parameters(), lr=0.001)
```

2) 모델 학습

drop out과 early stopping 동시 적용

```
epochs = 10
```

```
tr_loss_list, val_loss_list, val_acc_list = [], [], []
```



정훈이의 공부일지



```

best_loss = float('inf') # 초기값을 무한대로 설정
counter = 0

# 모델 저장 경로
PATH = './model.pt'

for t in range(epochs):
    tr_loss = train(train_dataloader, model, loss_fn, optimizer, device)
    val_loss, pred = evaluate(x_val, y_val, model, loss_fn, device)

    # accuracy 측정
    pred = nn.functional.softmax(pred, dim=1)
    pred = np.argmax(pred.cpu().numpy(), axis = 1)
    acc = accuracy_score(y_val.numpy(), pred)

    # 리스트에 추가
    tr_loss_list.append(tr_loss) # train - CrossEntropy
    val_loss_list.append(val_loss) # val - CrossEntropy
    val_acc_list.append(acc) # val - Accuracy

    print(f"Epoch {t+1}, train loss : {tr_loss:.4f}, val loss : {val_loss:.4f}, val acc : {acc:.4f}")

    # early stopping 확인 -----
    if val_loss > best_loss:
        counter += 1
        print(f"--> early stopping status, best_loss : {best_loss:.4f}")
    else:
        best_loss = val_loss
        counter = 0
        torch.save(model, PATH) # <- best model 저장하기

    # 조기 종료 조건 확인
    if counter >= patience:
        print("Early Stopping!")
        break
    # -----

```

```

Epoch 8, train loss : 1.3729, val loss : 1.1087, val acc : 0.6246
Epoch 9, train loss : 1.3520, val loss : 1.1012, val acc : 0.6216
Epoch 10, train loss : 1.3337, val loss : 1.0790, val acc : 0.6342

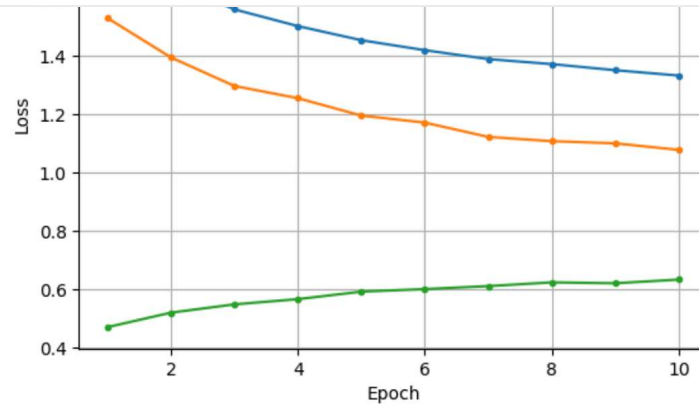
```

4) 모델평가

```
dl_learning_curve(tr_loss_list, val_loss_list, val_acc_list)
```



정훈이의 공부일지



5) 모델 선언수정

성능이 그렇게 좋지않게나와서 신경망을 cnn 더 깊이 쌓아보자

```
n_class=10
```

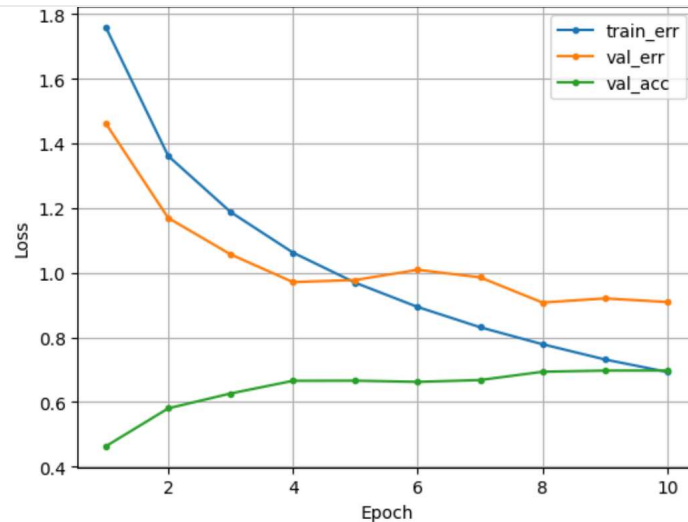
```
dropout_rate = 0.5
```

```
model = nn.Sequential(
    nn.Conv2d(3, 16, kernel_size=3, stride=1, padding=1),nn.ReLU(), # 3*32*32 -> 16*32*32
    nn.MaxPool2d(kernel_size=2), # 32*32*32 -> 32*16*16
    nn.Conv2d(16, 32, kernel_size=3, stride=1, padding=1),nn.ReLU(), # 16*16*16=> 32*16*16
    nn.MaxPool2d(kernel_size=2), # 32*16*16 => 32*8*8
    nn.Conv2d(32, 64, kernel_size=3, stride=1, padding=1),nn.ReLU(), #32*8*8 => 64*8*8
    nn.MaxPool2d(kernel_size=2), # 64*8*8=>64*4*4
    nn.Conv2d(64, 128, kernel_size=3, stride=1, padding=1),nn.ReLU(), #64*4*4=>128*4*4
    nn.MaxPool2d(kernel_size=2), # 128*2*2
    nn.Flatten(),
    nn.Linear(128*2*2, 256),nn.ReLU(),nn.Dropout(p=dropout_rate), # 128*2*2 => 256
    nn.Linear(256, n_class)
).to(device)
loss_fn =nn.CrossEntropyLoss()
optimizer = Adam(model.parameters(), lr =0.001)
```

```
Epoch 8, train loss : 0.7788, val loss : 0.9077, val acc : 0.6936
Epoch 9, train loss : 0.7316, val loss : 0.9206, val acc : 0.6972
--> early stopping status, best_loss : 0.907723
Epoch 10, train loss : 0.6929, val loss : 0.9094, val acc : 0.6974
--> early stopping status, best_loss : 0.907723
```



정훈이의 공부일지



모델을 복잡하게 할 수록 train 할 때 성능이 빠르게 수렴되는 것을 볼 수 있다.

6) dropout 추가

```
n_class=10
```

```
dropout_rate = 0.5
```

```
model = nn.Sequential(
    nn.Conv2d(3, 16, kernel_size=3, stride=1, padding=1),nn.ReLU(), # 3*32*32 -> 16*32*32
    nn.MaxPool2d(kernel_size=2), # 32*32*32 -> 32*16*16
    nn.Conv2d(16, 32, kernel_size=3, stride=1, padding=1),nn.ReLU(), # 16*16*16=> 32*16*16
    nn.MaxPool2d(kernel_size=2), # 32*16*16 => 32*8*8
    nn.Conv2d(32, 64, kernel_size=3, stride=1, padding=1),nn.ReLU(),nn.Dropout2d(p=dropout_rate),#32*8*8
    => 64*8*8
    nn.MaxPool2d(kernel_size=2), # 64*8*8=>64*4*4
    nn.Conv2d(64, 128, kernel_size=3, stride=1, padding=1),nn.ReLU(),nn.Dropout2d(p=dropout_rate),
    #64*4*4=>128*4*4
    nn.MaxPool2d(kernel_size=2), # 128*2*2
    nn.Flatten(),
    nn.Linear(128*2*2, 256),nn.ReLU(),nn.Dropout(p=dropout_rate), # 128*2*2 => 256
    nn.Linear(256, n_class)
).to(device)
loss_fn =nn.CrossEntropyLoss()
optimizer = Adam(model.parameters(), lr =0.001)
```

성능이 10 에포크로 좋지는 않지만 얼리스타핑도 안나고 과적합되지 않는 특징을 보였다.

7) dense linear 늘리기

```
n_class=10
```

```
dropout_rate = 0.5
```



정훈이의 공부일지



```

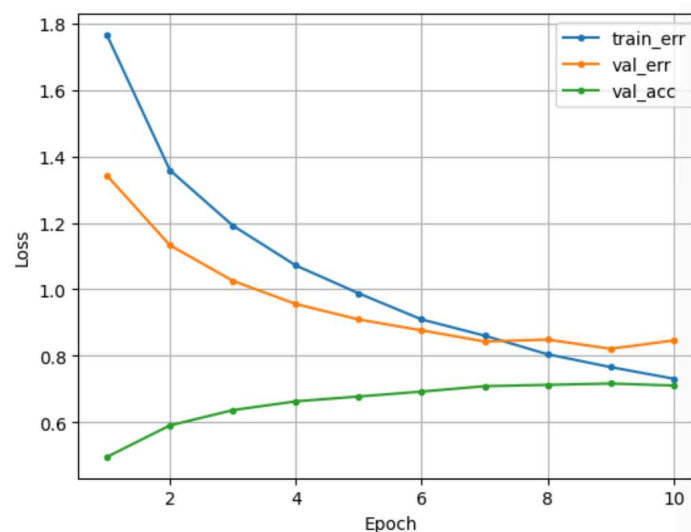
nn.MaxPool2d(kernel_size=2), # 32*32*32 -> 32*16*16
nn.Conv2d(32, 64, kernel_size=3, stride=1, padding=1), nn.ReLU(), # 32*16*16 => 64*16*16
nn.MaxPool2d(kernel_size=2), # 64*16*16 => 64*8*8
nn.Conv2d(64, 128, kernel_size=3, stride=1, padding=1), nn.ReLU(), # 64*8*8 => 128*8*8
nn.MaxPool2d(kernel_size=2), # 128*4*4
nn.Flatten(),
nn.Linear(128*4*4, 256), nn.ReLU(), nn.Dropout(p=dropout_rate), # 64*8*8 => 128
nn.Linear(256, 128), nn.ReLU(), nn.Dropout(p=dropout_rate),
nn.Linear(128, n_class)
).to(device)
loss_fn = nn.CrossEntropyLoss()
optimizer = Adam(model.parameters(), lr=0.001)
dense 쪽 신경망을 늘렸더니 더 성능이 좋아졌다.

```

```

Epoch 8, train loss : 0.8048, val loss : 0.8491, val acc : 0.7130
--> early stopping status, best_loss : 0.843328
Epoch 9, train loss : 0.7664, val loss : 0.8213, val acc : 0.7170
Epoch 10, train loss : 0.7315, val loss : 0.8465, val acc : 0.7110
--> early stopping status, best_loss : 0.821347

```



공감



정훈이의 공부일지



정훈이의 공부일지

개발하면서 겪는 시행착오와 배움을 기록합니다. 백엔드, AI, 데이터 분석, 프로젝트 경험 중심으로 운영 중입니다. 공부하면서 얻은 인사이트를 함께 나누고 싶어요!

댓글 0

endingo

내용을 입력하세요.

등록

공지사항

- 최근에 올라온 글
- [이미지처리] 5. YOLO 모델

[이미지처리] 3. CIFAR-10 분류...

[Algorithm] 2. 분리수거장 개념

[이미지처리] 2. CNN모델링2

- 최근에 달린 댓글
- 덕분에 좋은 정보 얻어 갑니다 ! ...

좋은 글 잘 보고 가요! 감사합니...

Total	
211	
Today	6
Yesterday	10

링크

- TAG
- git

github

[more](#)

« 2025/04 »						
일	월	화	수	목	금	토
		1	2	3	4	5
6	7	8	9	10	11	12
13	14	15	16	17	18	19
20	21	22	23	24	25	26
27	28	29	30			

- 글 보관함
- 2025/04 (24)

2025/03 (7)

2023/09 (14)



정훈이의 공부일지



AI와 엑셀, 데이터 분석을 좋아하는 개발자입니다.

다양한 문제 해결과 창의적인 개발을 추구합니다.

 이메일 보내기